网信办非沙箱文件过度访问排查指引

问题原文

App启动时，在荣耀手机上提示手机文件访问拦截记录和文件删除记录
App 在用户未授予存储权限的情况下，尝试读取非媒体影音文件

荣耀上文件的拦截记录和读取记录，基本跟非沙箱文件有关。

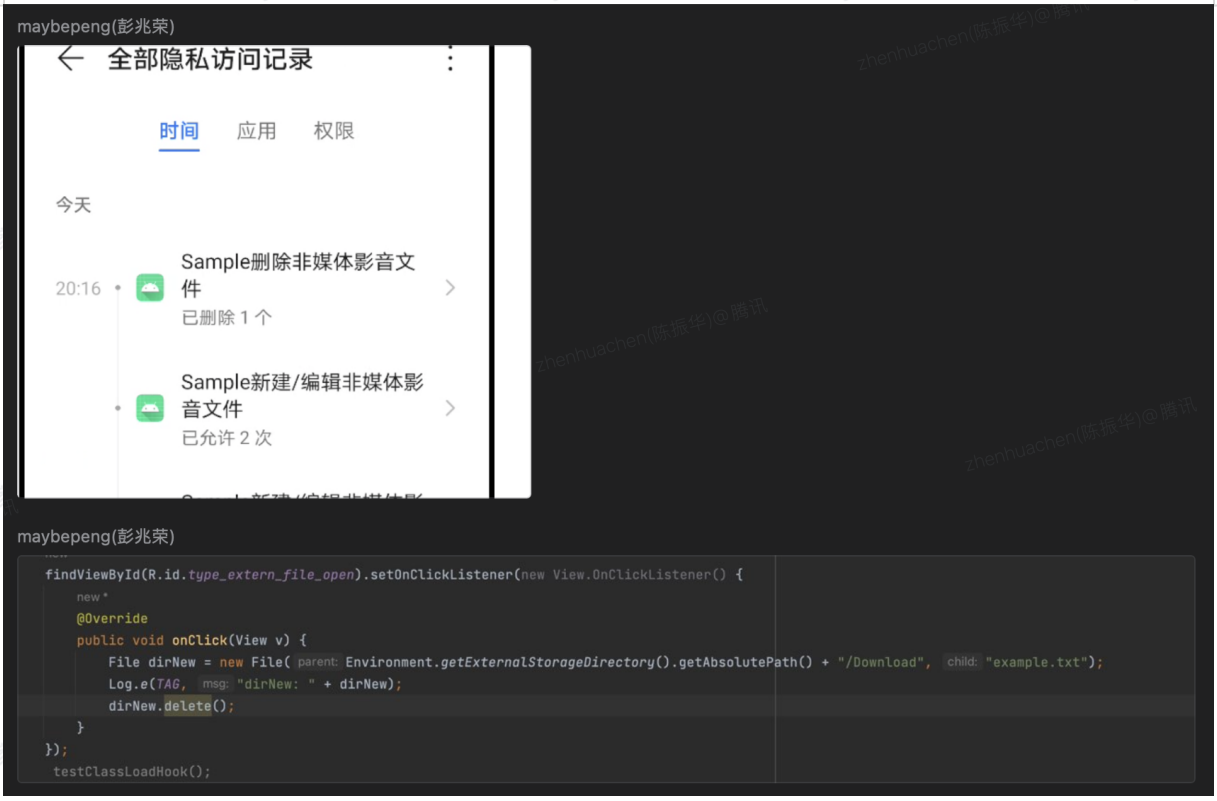
问题案例

1. mkdir 案例：以下隐私访问记录应该问0次

启动在xx目录上，调用mkdir

2、remove 案例

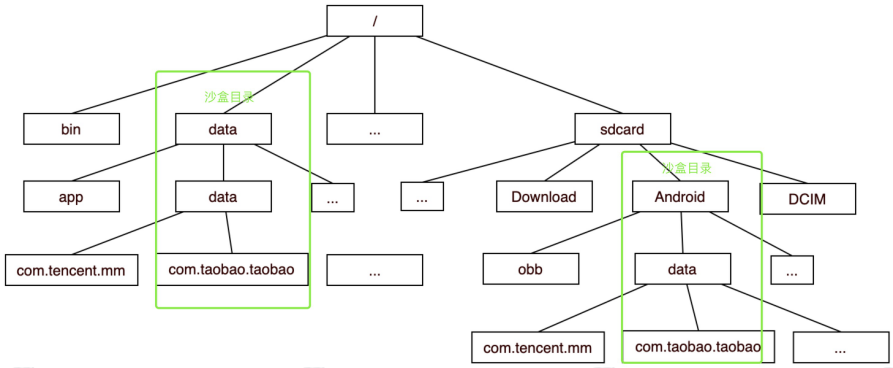
启动在Download目录上删除非影因文件



问题解读

- **过度访问原理**：无权限时或有权限无必要场景的情况下，使用权限相关功能API。
- **理解非沙箱文件：**

图中的绿框部分是应用私有路径，其他路径都算非沙箱文件。 Android 10开始，非沙箱文件都需要存储权限才能访问。



- **理解需要存储权限的路径：**Android的存储权限设计用于管控/sdcard或/storage/emluated/0为前缀的路径访问。（其它/system, /users等奇怪的开头不在管控范畴，可以先忽略）
- **过度访问存储权限原理**：访问**需要存储权限的文件路径**（非沙箱文件）。如果无权限或不符合功能预期，则以下可能算问题并且产生**问题记录**：
  1、**无存储权限**时对**需要存储权限的目录**做了任意操作（如createNewFile，mkdir，delete，FileReader，FileInputStream等）（**该部分问题记录是本次网信办整改重点**）
  2、**有存储权限**时对**需要存储权限的目录**做了**非必要访问**，启动删除了某个SD卡的txt文

件，不符合功能预期（**次要问题，建议应解尽解**）

3、~~有权限启动在无功能需要时读取SD卡中的相册文件等。~~

- **如何理解非影音**：影音目录通常是指DCIM和Pictures这两个，非影音则是其他文件。

Rightly检测工具

1、Rightly版本：

（mavens：https://mirrors.tencent.com/repository/maven/thirdparty-snapshots/ ）

版本变化：

0.9.23-rc13-SNAPSHOT（1、支持boot_id的访问排查 2、增加MediaStore的监控）

0.9.23-rc10-SNAPSHOT（解决堆栈倒序的问题）

0.9.23-rc9-SNAPSHOT（解决日志多次打，日志分开难以定位的问题）

0.9.23-rc7-SNAPSHOT （优化/sdcard私有目录未被过滤的问题，优化xunwind日志过大导致Logcat二次打印的问题）

0.9.23-rc6-SNAPSHOT（优化过滤条件，更全面发现问题，记得引入xunwind）

0.9.23-rc5-SNAPSHOT（新增对Native堆栈的打印支持，仅测试过Android12）

需额外引入unwind库，另外目前支持Android12机器，Android14机器暂不支持

implementation 'io.github.hexhacking:xunwind:2.0.0'

0.9.23-rc3-SNAPSHOT（扩充了C函数列表，详见“如何分析问题”）

2、先接入Rightly主SDK：https://docs.bugly.woa.com/rightly/dynamic/docs/Android/init

3、再接入Rightly扩展能力 - Bhook：

```
// build.gradle 接入以来：
implementation "com.tencent.qmethod.monitor:qmethod-native-bhook:0.9.23-rc2-SNAPSHOT"
implementation 'io.github.hexhacking:xunwind:2.0.0'

// 启动今早初始化Bhook
if (BuildConfig.DEBUG) {
    boolean isDebug = false;
    PMonitorBHook.init(this,
        new String[]{},  // ignore so files
        new int[]{433},  // ignore scoket port
        isDebug); //

}
```

4. 支持Native堆栈的输出（升级到0.9.23-rc5-SNAPSHOT）

如何分析问题

- **过滤日志：**

  0.9.23-rc2-SNAPSHOT版本

  message~=sdCardNoPermAccess（注：）

  0.9.23-rc3-SNAPSHOT版本

  vv~=SdcardAccess （所有SD卡访问日志）

  message~=SdcardNoPermWrite（严重问题，无权限写SD卡操作 或 读取SD卡文件内容操作，注！！：目前的分类有些小BUG，部分open函数可能是读场景，也会误识别成SdcardNoPermWrite。 但这类问题依然算是严重问题，建议业务专注治理SdcardNoPermWrite过滤出来的问题）

  message~=SdcardNoPermRead（待定问题）

  message~=SdcardHasPermWrite（待定问题，在启动场景大概率算问题。 其它相册路径（jpg结尾或目录包括DCIM，PICTURES）需要结合是否符合功能预期）

  message~=SdcardHasPermRead（待定问题，同上)

- **日志格式：**

```
method={Native方法}， {路径}
{堆栈}
```

- **怎么让日志更全面：**
  - **第一次启动卸载重装：**有些文件存在后不会再走创建，全新安装可以避免漏测
  - **第二次启动争取拉取到闪屏广告：**有条件才执行的代码，可能是你遗漏的噩梦
  - **第三次启动等等：**多思考下你的App还有哪些业务活动，多启动启动收集最全的日志

- **日志中哪些问题更严重**：（先关注写入操作，再关注读取操作）目前检测范围有点扩大，相当于把所有SD卡访问都纳入了打印，大家可以根据监管变化。

  1、严重问题 - 无权限时写入操作 或 读取文件内容（启动无权限就访问的严重问题，目前必改）

  查找字符串（method=open ） 创建文件 or 读取文件内容，对应Java的File.createNewFile 或 new FileInputStream(文件路径)等

  查找字符串（method=mkdir） 创建文件夹，对应Java的File.mkdir和File.mkdirs

  查找字符串（method=remove） 删除文件，对应Java的File.delete

  查找字符串（method=rmdir）删除文件夹，对应Java的File.delete

  查找字符串（method=fopen）创建文件

  避免误判注意：在使用相册功能中会大量的日志出现影音文件记录出现，目前检测还不支持识别文件类型，大家自行判断，建议先找出无存储权限下的问题。

  2、待定问题 -- 读取操作

  access 判断是否存在，常用于Java判断文件存在即可exist可能使用

  stat 获取文件状态 ，常用于Java判断文件存在即可exist可能使用

  opendir 打开文件 ，对应Java的listFile，算读取操作

  statvfs 获取文件信息，StatFs的Java类

  3、打印的日志图片，算影音文件，算问题吗

  建议结合场景来，如果符合预期则算，如果不符合预期则不算。

案例

```
SdCardNoPermAccess ---------------------------------
SdCardNoPermAccess: method=mkdir path=/storage/emulated/0/DCIM/AB
sdCardNoPermAccess: dalvik.system.VMStack.getThreadStackTrace(Native Method)
sdCardNoPermAccess: java.lang.Thread.getStackTrace(Thread.java:1734)
sdCardNoPermAccess: com.tencent.qmethod.bhookmonitor.PMonitorBHook.printJavaStackTrace(PMonitorBHook.java:163)
sdCardNoPermAccess: java.io.UnixFileSystem.createDirectory0(Native Method)
sdCardNoPermAccess: java.io.UnixFileSystem.createDirectory(UnixFileSystem.java:354)
sdCardNoPermAccess: java.io.File.mkdir(File.java:1323)
sdCardNoPermAccess: com.tencent.mobileqq.qmethodmonitor.MainApplication.onCreate(MainApplication.java:180)
sdCardNoPermAccess: android.app.Instrumentation.callApplicationOnCreate(Instrumentation.java:1225)
sdCardNoPermAccess: android.app.ActivityThread.handleBindApplication(ActivityThread.java:8583)
sdCardNoPermAccess: android.app.ActivityThread.access$2800(ActivityThread.java:311)
sdCardNoPermAccess: android.app.ActivityThread$H.handleMessage(ActivityThread.java:2889)
sdCardNoPermAccess: android.os.Handler.dispatchMessage(Handler.java:117)
sdCardNoPermAccess: android.os.Looper.loopOnce(Looper.java:205)
sdCardNoPermAccess: android.os.Looper.loop(Looper.java:293)
sdCardNoPermAccess: android.app.ActivityThread.loopProcess(ActivityThread.java:9934)
sdCardNoPermAccess: android.app.ActivityThread.main(ActivityThread.java:9923) <1 internal line>
sdCardNoPermAccess: com.android.internal.os.RuntimeInit$MethodAndArgsCaller.run(RuntimeInit.java:586)
sdCardNoPermAccess: com.android.internal.os.ZygoteInit.main(ZygoteInit.java:1240)
SdCardNoPermAccess ---------------------------------
SdCardNoPermAccess: method=open path=/storage/emulated/0/DCIM/AB/1.txt
sdCardNoPermAccess: dalvik.system.VMStack.getThreadStackTrace(Native Method)
```

解决效果排查

验证解决效果这里，可使用荣耀手机的隐私访问记录复测，直到0次为止。

（注：若手中无真机，可用公司云真机平台

https://cloudtest.woa.com/console/default-cl9d372a3989vkhm91cg/testlab/remote/devi...

选择Android12的荣耀手机，型号（FNE-AN00）进行测试）

已明确有问题的路径和堆栈

某安全能力

解决经验：升级

路径：/sdcard/Android/.android_lq

```
SdcardNoPermWrite: #00 pc 00000000000080dc  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libxunwind.so
#01 pc 0000000000007e8c  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libxunwind.so
#02 pc 000000000000090ec  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libxunwind.so (xunwind_cfi_get+88)
#03 pc 000000000000089bc  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libxunwind.so
#04 pc 0000000000222244  /apex/com.android.art/lib64/libart.so (art_quick_generic_jni_trampoline+148)
#05 pc 00000000000218be8  /apex/com.android.art/lib64/libart.so (art_quick_invoke_static_stub+568)
#06 pc 00000000002845c4  /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int, art::JValue*, char const*)+216)
#07 pc 00000000003fd69c  /apex/com.android.art/lib64/libart.so (art::interpreter::ArtInterpreterToCompiledCodeBridge(art::Thread*, art::ArtMethod*, art::ShadowFrame*, unsigned short
#08 pc 00000000003f8510  /apex/com.android.art/lib64/libart.so (bool art::interpreter::DoCall<false, false>(art::ArtMethod*, art::Thread*, art::ShadowFrame&, art::Instruction const
#09 pc 00000000007a0eb4  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+980)
#10 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
#11 pc 00000000002ab4e8  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/split_lib_dependencies5_apk.apk (offset 0x27ca000) (io.github.hexhacking
#12 pc 00000000007a1348  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+2152)
#13 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
#14 pc 00000000002ab548  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/split_lib_dependencies5_apk.apk (offset 0x27ca000) (io.github.hexhacking
#15 pc 00000000007a1348  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+2152)
#16 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
#17 pc 00000000001d8da8  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/split_lib_dependencies3_apk.apk (offset 0xaae000) (com.tencent.qmethod.b
#18 pc 00000000007a1348  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+2152)
#19 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
#20 pc 00000000001d904c  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/split_lib_dependencies3_apk.apk (offset 0xaae000) (com.tencent.qmethod.b
#21 pc 00000000003effa8  /apex/com.android.art/lib64/libart.so (art::interpreter::Execute(art::Thread*, art::CodeItemDataAccessor const&, art::ShadowFrame&, art::JValue, bool, bool
#22 pc 0000000000077cd9c  /apex/com.android.art/lib64/libart.so (artQuickToInterpreterBridge+776)
#23 pc 0000000000222378  /apex/com.android.art/lib64/libart.so (art_quick_to_interpreter_bridge+88)
#24 pc 00000000000218be8  /apex/com.android.art/lib64/libart.so (art_quick_invoke_static_stub+568)
#25 pc 00000000002845c4  /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int, art::JValue*, char const*)+216)
#26 pc 000000000006556e0  /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs<art::ArtMethod*>(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, art::A
#27 pc 0000000000655ba4  /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs<_jmethodID*>(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, _jmethodID*
#28 pc 000000000051aaa4  /apex/com.android.art/lib64/libart.so (art::JNI<true>::CallStaticVoidMethodV(_JNIEnv*, _jclass*, _jmethodID*, std::__va_list)+640)
#29 pc 00000000001ecf8  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libpmnaitvemonitor.so
#30 pc 00000000001a2f0  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libpmbhookmonitor.so (_JNIEnv::CallStaticVoidMethod(_jclass*,
#31 pc 00000000001c300  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libpmbhookmonitor.so (callbackToJava(char const*, char const*)+
#32 pc 00000000001d8d8  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libpmbhookmonitor.so
#33 pc 0000000000892ebc  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libfekit.so
#34 pc 0000000000249ef4  /data/app/~~sTCMDMFuYcoY3b336gIwpw==/com.tencent.mobileqq-R0d36m_evnax1Cd6vfC_Jw==/lib/arm64/libfekit.so
 ---------------------------------
```

turing 图灵盾

解决经验：图灵盾除了SDK自身，在广告SDK上还有一份拷贝（名字不同）。即需要同时升级图灵盾和广告SDK，避免问题被发现

method=fopen path=/storage/emulated/0/.turing.dat

```
SdcardNoPermWrite method=fopen
path=/storage/emulated/0/.turing.dat
stack=#21 pc 000000000001faec  /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA==/lib/arm64/libturingga.so
#22 pc 000000000001f44c  /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA==/lib/arm64/libturingga.so
#23 pc 0000000000002384c  /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA==/lib/arm64/libturingga.so
#26 pc 000000000040d786  [anon:dalvik-classes10.dex extracted in memory from /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA=
.tencent.turingfd.sdk.ams.ga.Filbert.c+178)
#28 pc 0000000000412f74  [anon:dalvik-classes10.dex extracted in memory from /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA=
.tencent.turingfd.sdk.ams.ga.TuringSDK.init+196)
#30 pc 000000000049ca24  [anon:dalvik-classes5.dex extracted in memory from /data/app/~~ePxOSE_p-Ux6pPXWhu9uKw==/com.tencent.map-IjDNXtcj54ppn_JrRI9eQA==
```

SuperPlayer视频SDK

解决：联系视频SDK团队进行升级，记得打包正确的版本

method=open path=/sdcard/tencent/tmp//vodPcdnMSdkPeerId.log

```
cess: SdcardNoPermWrite: #00 pc 00000000000080dc  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libxunwind.so
cess: #01 pc 00000000000007e8c  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libxunwind.so
cess: #02 pc 00000000000090ec  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libxunwind.so (xunwind_cfi_get+88)
cess: #03 pc 00000000000089bc  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libxunwind.so
cess: #04 pc 0000000000222244  /apex/com.android.art/lib64/libart.so (art_quick_generic_jni_trampoline+148)
cess: #05 pc 00000000000218be8  /apex/com.android.art/lib64/libart.so (art_quick_invoke_static_stub+568)
cess: #06 pc 0000000002847d4  /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int, art::JValue*, char const*)+216)
cess: #07 pc 00000000003ecf50  /apex/com.android.art/lib64/libart.so (art::interpreter::ArtInterpreterToCompiledCodeBridge(art::Thread*, art::ArtMethod*, art::ShadowFrame*, unsigned short, art::JVal

cess: #08 pc 00000000003e7dc4  /apex/com.android.art/lib64/libart.so (bool art::interpreter::DoCall<false, false>(art::ArtMethod*, art::Thread*, art::ShadowFrame&, art::Instruction const*, unsigned

cess: #09 pc 000000000076e0a0  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+980)
cess: #10 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
cess: #11 pc 00000000003be05c  [anon:dalvik-classes6.dex extracted in memory from /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/base.apk!classes6.dex:00000071b1981000

cess: #12 pc 000000000076e534  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+2152)
cess: #13 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
cess: #14 pc 00000000003be0bc  [anon:dalvik-classes6.dex extracted in memory from /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/base.apk!classes6.dex:00000071b1981000
urrentThread+4)
cess: #15 pc 000000000076e534  /apex/com.android.art/lib64/libart.so (MterpInvokeStatic+2152)
cess: #16 pc 0000000000203994  /apex/com.android.art/lib64/libart.so (mterp_op_invoke_static+20)
cess: #17 pc 0000000000475b64  [anon:dalvik-classes10.dex extracted in memory from /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/base.apk!classes10.dex:00000071afbbc0
ook.getStack)
cess: #18 pc 0000000003dfba8  /apex/com.android.art/lib64/libart.so (art::interpreter::Execute(art::Thread*, art::CodeItemDataAccessor const&, art::ShadowFrame&, art::JValue, bool, bool)+304)
cess: #19 pc 0000000000749b6c  /apex/com.android.art/lib64/libart.so (artQuickToInterpreterBridge+776)
cess: #20 pc 0000000000222378  /apex/com.android.art/lib64/libart.so (art_quick_to_interpreter_bridge+88)
cess: #21 pc 00000000021eb828  /memfd:jit-cache (deleted) (offset 0x2000000) (com.tdsrightly.qmethod.bhookmonitor.PMonitorBHook.printJavaStackTrace+1016)
cess: #22 pc 00000000000218ba8  /apex/com.android.art/lib64/libart.so (art_quick_invoke_static_stub+568)
cess: #23 pc 0000000002847d4  /apex/com.android.art/lib64/libart.so (art::ArtMethod::Invoke(art::Thread*, unsigned int*, unsigned int, art::JValue*, char const*)+216)
cess: #24 pc 0000000062df0c  /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs<art::ArtMethod*>(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, art::ArtMethod*,

cess: #25 pc 0000000062e3d0  /apex/com.android.art/lib64/libart.so (art::JValue art::InvokeWithVarArgs<_jmethodID*>(art::ScopedObjectAccessAlreadyRunnable const&, _jobject*, _jmethodID*,

cess: #26 pc 0000000000503ec8  /apex/com.android.art/lib64/libart.so (art::JNI<true>::CallStaticVoidMethodV(_JNIEnv*, _jclass*, _jmethodID*, std::__va_list)+616)
cess: #27 pc 00000000001a2f0  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libpmbhookmonitor.so (_JNIEnv::CallStaticVoidMethod(_jclass*, _jmethodID*, ...)
cess: #28 pc 000000000001c300  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libpmbhookmonitor.so (callbackToJava(char const*, char const*)+940)
cess: #29 pc 000000000001d8d8  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libpmbhookmonitor.so
cess: #30 pc 00000000079fcc8  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #31 pc 00000000079f89c  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #32 pc 00000000079f43c  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #33 pc 00000000079e950  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #34 pc 00000000079bcec  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #35 pc 00000000079bab4  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
cess: #36 pc 00000000079f9ec  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (teg_pcdn::CreatePcdn(unsigned int)+52)
cess: #37 pc 00000000005ede08  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::PcdnManager::ObtainPcdnInstance(int)+292)
cess: #38 pc 00000000005ebd30  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::PcdnDownloader::init(int)+96)
cess: #39 pc 00000000005ebeac  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::PcdnDownloader::SendRequest(int, int, int,
ar_traits<char>, std::__ndk1::allocator<char> > const&, long, long, int, std::__ndk1::basic_string<char, std::__ndk1::char_traits<char>, std::__ndk1::allocator<char> > const&, int, int)+104)
cess: #40 pc 00000000063ef04  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
oxy::PcdnDownloader*, int, int)+1208)
cess: #41 pc 00000000061dc00  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::IScheduler::TryPcdnDownload(int)+80)
cess: #42 pc 00000000005fb380  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::FileVodHttpScheduler::FastDownload()+152)
cess: #43 pc 00000000000625a30  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::IScheduler::OnMDSEComplete(tpdlproxy::MDSECallb

cess: #44 pc 0000000000625514  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
tpdlproxy::MDSECallback&)+608)
cess: #45 pc 0000000000622414  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlproxy::IScheduler::OnHandleMDSECallBack(void*, void*,

cess: #46 pc 0000000000642e24  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlpubliclib::TimerT<tpdlproxy::IScheduler>::OnEvent()+17
cess: #47 pc 00000000005bdf30  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlpubliclib::TimerThread::HandleEvent()+60)
cess: #48 pc 00000000005bdd30  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so (tpdlpubliclib::TimerThread::TimerProc(void*, void*)+96)
cess: #49 pc 00000000005bdcc0  /data/app/~~etDWID4PVYMoyfIcIlkxPA==/com.tencent.mtt-r-f1USYy9pvWKMfu3mDLnQ==/lib/arm64/libDownloadProxy.so
ead>::ThreadProc()+56)
```

QQ互联SDK

解决方案：升级SDK

路径：method=open path=/storage/emulated/0/Tencent/msflogs/com/tencent/mobileqq/com.tencent.mobileqq_connectSdk.24.04.29.17.log

```
04-29 17:22:35.932 22031 15765 E PandoraEx.NetworkManager: SdCardNoPermAccess ---------------------------------
04-29 17:22:35.932 22031 15765 E PandoraEx.NetworkManager: SdCardNoPermAccess: method=open path=/storage/emulated/0/Tencent/msflogs/com/tencent/mobileqq/
com.tencent.mobileqq_connectSdk.24.04.29.17.log
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: dalvik.system.VMStack.getThreadStackTrace(Native Method)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: java.lang.Thread.getStackTrace(Thread.java:1734)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: com.tencent.qmethod.bhookmonitor.PMonitorBHook.printJavaStackTrace(SourceFile:2)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: libcore.io.Linux.open(Native Method)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: libcore.io.ForwardingOs.open(ForwardingOs.java:567)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: libcore.io.BlockGuardOs.open(BlockGuardOs.java:273)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: libcore.io.ForwardingOs.open(ForwardingOs.java:567)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: android.app.ActivityThread$AndroidOs.open(ActivityThread.java:8918)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: libcore.io.IoBridge.open(IoBridge.java:561)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: java.io.FileOutputStream.<init>(FileOutputStream.java:236)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: java.io.FileWriter.<init>(FileWriter.java:107)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: m.t.x.i.d.p(SourceFile:7)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: m.t.x.i.d.o(SourceFile:5)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: m.t.x.i.d.handleMessage(SourceFile:2)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: android.os.Handler.dispatchMessage(Handler.java:110)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: android.os.Looper.loopOnce(Looper.java:206)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: android.os.Looper.loop(Looper.java:296)
04-29 17:22:35.932 22031 15765 E sdCardNoPermAccess: sdCardNoPermAccess: android.os.HandlerThread.run(HandlerThread.java:67)
```

业务解决经验

腾讯会议

结合我们对于安卓存储机制的了解，以及荣耀提供的非媒体影音文件说明，我们扩充如下：

腾讯会议（targetSDKversion 33）三个方向排查未授予存储权限的情况下尝试读取非媒体影音文件（荣耀magic7.0）

1. 检查应用是否有访问外部存储非APP私有目录（需要read_external_storage或read_meida_audio/images/video等存储权限才能访问到的目录）， 如果有，需改为判断是否有存储权限，有存储权限后再访问；

2.检查sdk是否有访问外部存储中非APP私有目录或非SDK私有目录（SDK可能不使用APP的私有目录，使用自己的私有目录）， 如果有则推进sdk处理， 处理方式同1；

3.已知图灵盾SDK会在外部存储上非APP/SDK私有目录写一个隐藏文件， 但在最新版本已修复， 建议更新图灵盾sdk。（手机内首次引入图灵盾SDK，该SDK会在外部存储上非APP/SDK私有目录上写一个隐藏文件，若手机内已存在腾讯系APP并使用了图灵盾SDK，则再安装其他使用了图灵盾SDK的APP安装可能不会被记录到未授予存储权限的情况下尝试读取非媒体影音文件，因为隐藏文件已创建会共用）

腾讯会议本身做过1次将存储由外部存储非APP私有目录调整为私有目录的大整改，因此可能涉及到的调整较小。